LAB TASK 7

1) MST implementation using Kruskal's Algorithm

CODE:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int findParent(int parent[], int node) {
    if (parent[node] == node)
        return node;

    return parent[node] = findParent(parent, parent[node]);
}

void unionSet(int parent[], int rank[], int u, int v) {
    u = findParent(parent, u);
    v = findParent(parent, v);

    if (rank[u] < rank[v])
        parent[u] = v;
    else if (rank[u] > rank[v])
```

```
        parent[v] = u;
    else {
        parent[v] = u;
        rank[u]++;
    }
}

void kruskal(int V, vector<Edge> edges) {

    sort(edges.begin(), edges.end(), compare);

    int parent[V];
    int rank[V];

    for (int i = 0; i < V; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    int totalCost = 0;

    cout << "Edges in MST:\n";

    int count = 0;

    for (Edge edge : edges) {

        int u = findParent(parent, edge.u);
        int v = findParent(parent, edge.v);

        if (u != v) {
```

```
            cout << edge.u << " - "
                 << edge.v << " : "
                 << edge.weight << endl;

            totalCost += edge.weight;

            unionSet(parent, rank, u, v);

            count++;

            if (count == V - 1)
                break;
        }
    }

    cout << "Total Cost = " << totalCost << endl;
}

int main() {

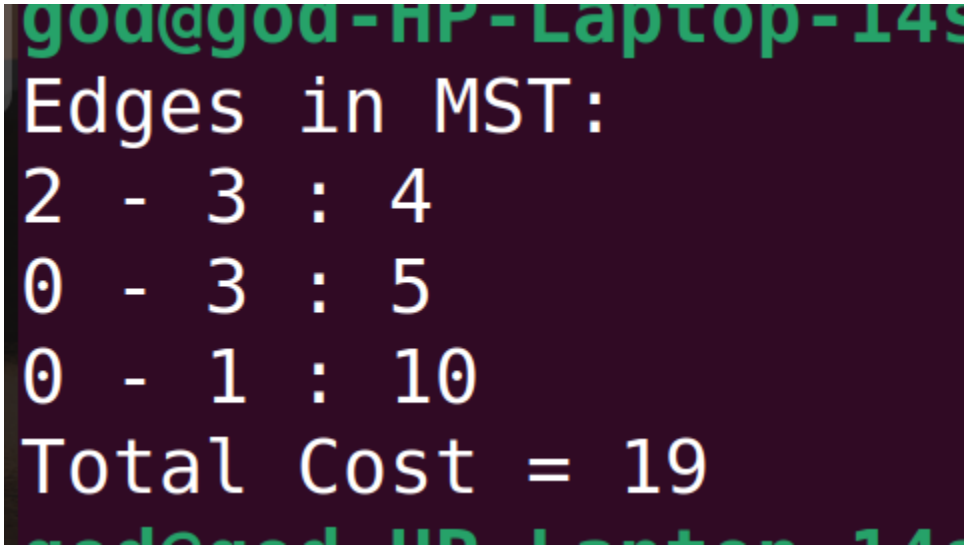
    int V = 4;

    vector<Edge> edges = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    };

    kruskal(V, edges);
```

```
    return 0;
}
```

OUTPUT:



2) MST implementation using Prim's Algorithm

Code:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

void prim(int graph[4][4], int V) {

    int parent[V];
    int key[V];
    bool visited[V];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        visited[i] = false;
```

```
}

key[0] = 0;
parent[0] = -1;

for (int count = 0; count < V - 1; count++) {

    int min = INT_MAX;
    int u;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && key[i] < min) {
            min = key[i];
            u = i;
        }
    }

    visited[u] = true;

    for (int v = 0; v < V; v++) {

        if (graph[u][v] != 0 &&
            !visited[v] &&
            graph[u][v] < key[v]) {

            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}
```

```
    int totalCost = 0;

    cout << "Edges in MST:\n";

    for (int i = 1; i < V; i++) {

        cout << parent[i] << " - "
             << i << " : "
             << graph[i][parent[i]] << endl;

        totalCost += graph[i][parent[i]];
    }

    cout << "Total Cost = " << totalCost << endl;
}

int main() {

    int graph[4][4] = {

        {0, 10, 6, 5},
        {10, 0, 0, 15},
        {6, 0, 0, 4},
        {5, 15, 4, 0}
    };

    prim(graph, 4);

    return 0;
```

}

OUTPUT:

